

FAG arch35 非 TND 确定性 BAND 三模式调度（算子同步版）

版本：v3.1 日期：2026-08-17

对应实现：SelectDeterBandSchedule / CalBandDeterIndex / CalDeterMaxLoopNum

1. 结论

分支不是按 Causal、Dense、Band 的固定几何优先级选择。Host 先构造 BAND 基线，再依次评估 DENSE 和 CAUSAL；候选轮次只有严格小于当前最优轮次时才替换。平局保持先前模式。

```
baseline = BAND
if denseRound < bestRound: mode = DENSE
if causalRound < bestRound: mode = CAUSAL
```

2. 外围入口条件

本方案只描述已经进入确定性 DETER_BAND 调度选择后的三模式。算子外围还要求 enableSwizzle、非 TND、splitAxis=BN2GS1S2、两侧 Cube 基本块相等、sparseType 可支持。

```
isDeterministic
deterSparseType == DETER_BAND
g == 1
coreNum == 2 * aicNum
actualBatch = (b-tailZeroCount)*n2 > 0
```

2.1 sparseMode=3 的远端兼容优先级

RIGHT_DOWN_CAUSAL 已满足远端原 swizzle 全部条件时，直接保留旧路径，不进入三模式选择器。Host 保持调度模式 DISABLED，且 isSplitByBlockIdx=true。Kernel 继续调用 CalCausalSwizzleIndex 和配套旧轮次公式，避免重新选择为 DENSE 后发生性能回退。

```
legacyRightDown = sparseMode==3 and oldIsSplitByBlockIdx
if legacyRightDown:
    scheduleMode=DISABLED; isSplitByBlockIdx=true
    kernelDispatch=CalCausalSwizzleIndex
else:
    evaluate BAND / DENSE / CAUSAL
```

旧入口还要求原始 $b*n2$ 为偶数。若示例 $B=17$ 是扣除 tailZero 后的 actualBatch，而原始 Batch 为 18，则旧路径仍成立；若原始 $b*n2$ 本身就是 17，远端条件不会开启该 swizzle。 $m=33$ 、 $n=49$ 时旧公式使用 $causalM=33$ 、 $causalN=68$ 。

3. 参数归一化

```

p = min(p,m); q = min(q,n)
if p < 0: (m,n,p,q) = (m,n+p,1,p+q)
if q < 0: (m,n,p,q) = (m+q,n,p+q,1)

p=min(p,m); q=min(q,n)
m=min(m,n+p-1); n=min(n,m+q-1)

```

Python 的 `normalize_deter_band_schedule_params()` 与 `seg_params()` 按相同顺序执行，负 token 转换、clamp 和无效空行列裁剪均与算子一致。

4. BAND 基线

根据 $p+q$ 与 m 的关系计算 $L1/L2/L3$ 、有效块数 `bandBlocks`、可配对列数 `hybridPairCount`，以及每个 Batch 的虚拟列数。

```

colsPerBatch = L1+L2+L3-hybridPairCount
bandSlot    = p+q-1
bandRound   = ceil(b*colsPerBatch/k)*bandSlot

```

5. DENSE 候选

```

denseK = min(k,m*b)
eligible = denseK==k and min(denseK,n)<=m
denseRound = ceil(n*b/denseK)*m
仅当 denseRound < bestRound 时选择 DENSE

```

因此 DENSE 与 BAND 轮次相等时仍保持 BAND，这正是旧 Python 与算子容易不一致的边界。

6. CAUSAL 候选

当前算子只采用 lower causal embedding。upperWaste 即使更小也不会转置使用，因为会破坏原始列固定归核约束。

```

lowerSize = m+q-1
upperSize = n+p-1
lowerWaste = C(lowerSize)-bandBlocks
useLower = bandBlocks>0 and 0<=lowerWaste<=upperWaste
           and lowerWaste<=floor((bandBlocks-1)/10)

```

CAUSAL 还要求至少存在一个 Batch pair、配对段不减少活跃核数，并满足 $\text{causalK} \leq \text{lowerSize}$ 。

```

pairCount = b//2
causalK   = min(k,lowerSize*pairCount)
eligible   = pairCount>0 and causalK==k and causalK<=lowerSize
pairRound = ceil((lowerSize+1)*pairCount/causalK)*lowerSize

```

6.1 奇数尾 Batch

b 为奇数时，前 $b-1$ 个 Batch 继续成对拼接；最后一个 Batch 调用 `CalCausalSingleBatchDeterIndex`。该函数保持同行同轮互斥和整列单核归属。

```

groupCount = lowerSize//(2*k)
groupRound = (2*lowerSize+1)*groupCount-2*k*groupCount^2
remain = lowerSize-2*k*groupCount
tailRound = groupRound + (remain if remain<=k
                        else max(remain,2*remain-2*k+1))
causalRound = pairRound+tailRound

```

7. 目标 case 的实际分支

```

k,m,n,p,q,b = 28,55,55,55,1,13
BAND = 1430
DENSE = 1430 (平局, 不切换)
CAUSAL pair = 660
CAUSAL tail = 55
CAUSAL total = 715

```

最终 mode=CAUSAL, maxRound=715。有效任务数为 20,020, 28×715 也为 20,020; 每核负载 715, 逻辑浪费轮次为 0。

8. Kernel 坐标分发

```

r <= pairRound:
    CalCausalSwizzleIndex(..., pairedBatch, r)
r > pairRound and b is odd:
    CalCausalSingleBatchDeterIndex(..., r-pairRound)
    batchId = b
最后执行 x -= q-1 和 BAND 有效区过滤

```

Kernel 的 CAUSAL 总轮次直接读取 Host 序列化的 deterMaxRound; pairRound 在 Kernel 按同一公式计算, 因此阶段边界和总轮次均与 Host 对齐。

9. Python 验证项

验证包括: 分支和 maxRound、坐标全覆盖、无重复/越界、maxRound 后无有效任务、同 Batch 同行同轮无冲突、同一实际列不跨核、偶数配对段和奇数尾段的边界连续。

```

python band_hybrid.py --full-test
python band_hybrid.py --case 28 55 55 55 1 13
python band_hybrid.py --render-pdf band_hybrid.pdf

```

10. 回退说明

Python 的 DISABLED 只表示三模式选择器未启用; 完整算子还会根据 isSplitByBlockIdx 和原有确定性路径回退。脚本的 k/m/n/p/q/b 分支判断以外围条件已经满足为前提。